Distributed PageRank with MPI+OpenMP and MPI+CUDA

DSA5208 Project 3 · November 2026

Ma Xiaoxiao A0259173X
Ye Sifan A0262394Y
Chen Yuda A0353182H

Abstract

We parallelise the PageRank power iteration on sparse directed graphs of up to 65.6 million vertices and 3.6 billion edges. The main implementation is MPI+OpenMP: one MPI rank per NUMA domain, OpenMP threads inside the rank, a 1D partition of the in-edge matrix and a ghost-vertex exchange with `MPI_Neighbor_alltoallv`. A CUDA single-GPU version and an MPI+CUDA multi-GPU version complete the study. All timings were measured on the NUS Vanda cluster on up to 10 nodes (720 CPU cores) and on NVIDIA A40 GPUs. On com-friendster the MPI+OpenMP code reaches a strong-scaling speedup of 159 on 432 cores (25335 ms → 159 ms per iteration). On the smaller soc-LiveJournal1 graph, communication stops scaling at one node. Pure MPI with one rank per core breaks down above 144 ranks because inter-node traffic on Vanda goes over TCP: at 432 cores it is 54× slower than the hybrid code. One A40 GPU is about as fast as one 36-core CPU socket, because both are limited by random memory accesses and not by arithmetic.

Table 1. Where each grading item is addressed, and the main results.

Requirement	Section and main result
Algorithm explanation	§2: pull-based PageRank as an SpMV on an in-edge CSR, its 1D-partitioned MPI+OpenMP version and the CUDA kernels.
Serial runtime vs. edges and vertices	§6: time is linear in M (4.5 ms to 9.0 s); 2.3–7 ns per edge depending on whether x fits in cache.
MPI strong and weak scaling (≥256 cores), speedup, efficiency, communication	§7: up to 720 cores; com-friendster speedup 159 on 432 cores; weak efficiency 0.04 at 720 cores; the ghost exchange over TCP limits scaling, and pure MPI is 54× slower than hybrid at 432 cores.
MPI+X with OpenMP and a GPU	§8: OpenMP reaches 170 GB/s on one socket; one A40 GPU (5.42 ms) is about as fast as one 36-core socket; MPI+CUDA on two GPUs gives 1.68× on com-friendster.
Discussion	§9: findings on web-Google, how the work was parallelised, which parts communicate most, limitations.
Terminology, sparsity, synthetic data, validation	§1 (Table 2), §3, §5: every parallel variant matches the serial program to rounding error.

1 Problem and terminology

PageRank assigns every vertex v of a directed graph $G = (V, E)$, $N = |V|$, $M = |E|$, the stationary probability of a random surfer. With probability $d = 0.85$ the surfer follows a random out-edge, and otherwise it jumps to a uniformly random vertex. A *dangling* vertex has no out-edges; its probability mass is spread uniformly over all vertices. One iteration computes

$$r_v^{(k+1)} = \frac{1-d}{N} + d\,\frac{\sum_{u\in D} r_u^{(k)}}{N} + d\sum_{(u,v)\in E} \frac{r_u^{(k)}}{\mathrm{outdeg}(u)}, \tag{1}$$

starting from $r_v^{(0)} = 1/N$, where D is the set of dangling vertices. The iteration stops when the residual $\|r^{(k+1)} - r^{(k)}\|_1$ falls below a tolerance, or after a fixed number of iterations.

Table 2. Terminology and notation.

Term	Meaning
CSR / in-edge CSR	The graph as the sparse $N \times N$ matrix A with $A_{vu} = 1$ for each edge $u \to v$. An offset array `off` (N+1 entries) says where the in-edges of each vertex start in an index array `src` (M entries) of source vertices. Equation (1) is then a sparse matrix–vector product (*SpMV*) $y = Ax$ with $x_u = r_u/\text{outdeg}(u)$.
Pull formulation	Each vertex reads ("pulls") the contributions x_u of its in-neighbours and writes only its own r_v, so no atomic updates are needed (unlike a push formulation).
Rank, thread, core	An MPI *rank* is one process; it runs OpenMP *threads*, each pinned to one CPU *core*. "p cores" always means ranks $\times$ threads $= p$.
NUMA domain	One CPU socket of a Vanda node and its local memory (2 domains of 36 cores per node).
Pure MPI / hybrid	*Pure MPI*: one rank per core, one thread each. *Hybrid* (MPI+OpenMP): one rank per NUMA domain with 36 threads; below 36 cores, one rank with p threads.
Partition	The rows of A are split into P contiguous blocks, one per rank (*1D row partition*). *Vertex* partition: equal vertex counts. *Edge* partition: equal in-edge counts.
Owned / ghost vertex	A rank *owns* the vertices of its block. A *ghost* is a vertex owned by another rank that is the source of an in-edge of the block; its x_u must be received every iteration.
Allgather / ghost exchange	Two ways to deliver remote x values: `MPI_Allgatherv` gives every rank all of x; the ghost exchange sends only the ghost entries with `MPI_Neighbor_alltoallv`.
Load imbalance	$\max_i m_i/\bar{m}$, with m_i the in-edges of rank i and $\bar{m} = M/P$.
Strong / weak scaling	Strong: fixed graph, more cores. Weak: the graph grows with the core count (about 2 million edges per core).
Speedup, efficiency	$S_p = T_1/T_p$ and $E_p = S_p/p$, with T_p the time per iteration on p cores and T_1 the MPI+OpenMP program on one core (within 5% of the serial program). Weak-scaling efficiency: T_1/T_p.
t_{comp}, t_{comm}, t_{wait}	Time in local computation and in MPI calls, each summed over iterations and maximised over ranks (so they need not add up to T_p). In `-split` runs a barrier precedes every MPI call; its time is t_{wait} (imbalance and synchronisation), and t_{comm} is then pure data movement.
Effective bandwidth	$B = (12M + 44N)/T_p$: the minimum bytes one iteration reads and writes (4-byte index and 8-byte x value per edge; offsets, r, x, outdegree and output per vertex).
BTL, TCP, RDMA	OpenMPI's *byte transfer layer*: `vader` (shared memory), `tcp` (kernel sockets) or `openib` (RDMA, remote direct memory access by the network card without the operating system).
R-MAT	Recursive-matrix random graph generator with power-law degrees (Section 3).

The problem is not embarrassingly parallel. Every iteration needs values r_u owned by other processes, and two global sums (the dangling mass and the residual) on which every process must agree before the next iteration.

The terms used in the rest of the report are defined in Table 2.

2 Algorithm

Serial algorithm. The graph is read once from a binary in-edge CSR file. Each iteration then performs

1. **Contributions.** $x_u = r_u/\text{outdeg}(u)$ for every vertex; the dangling mass $\delta = \sum_{u \in D} r_u$ is accumulated at the same time ($O(N)$).
2. **SpMV.** For every vertex v: $s_v = \sum_{e=\text{off}_v}^{\text{off}_{v+1}-1} x_{\text{src}_e}$ and $r'_v = (1-d)/N + d\,\delta/N + d\,s_v$ ($O(M)$). The residual $\sum_v |r'_v - r_v|$ is accumulated in the same loop.
3. **Swap** $r \leftarrow r'$ and test the residual against the tolerance.

The work per iteration is $\Theta(N + M)$ with about two floating-point operations per edge. The SpMV reads x_{src_e} in the order of the edge list, which for a power-law graph is close to random. Each such read moves a whole 64-byte cache line (CPU) or a 32-byte sector (GPU) to deliver 8 useful bytes.

PageRank is therefore limited by memory bandwidth and latency, not by floating-point throughput: an iteration does 0.17 floating-point operations per byte. Every analysis below is phrased in those terms.

Distributed iteration (MPI+OpenMP). Rank i owns rows $[\ell_i, h_i)$ of A and the matching entries of r. One iteration is

1. compute the owned x_u and the local dangling mass (OpenMP `parallel for`, static schedule);
2. **communication 1:** deliver the remote x values, either with `MPI_Allgatherv` (all of x) or with the ghost exchange (pack the entries that other ranks need, then `MPI_Neighbor_alltoallv`);
3. **communication 2:** `MPI_Allreduce` of the dangling mass (one double);
4. local SpMV over the owned rows (OpenMP `parallel for`, `schedule(dynamic,512)`, because row lengths follow a power law);
5. **communication 3:** `MPI_Allreduce` of the residual (one double).

The ghost plan is built once during setup. Each rank collects the distinct source vertices of its in-edges that lie outside its block, groups them by owner, and renumbers the column indices into a local index space [owned | ghosts], so the SpMV loop is identical for both exchange methods. Only rank 0 reads the offset array to compute the partition bounds; every rank then reads its own slice of the CSR file. MPI is initialised with `MPI_THREAD_FUNNELED`: only the master thread calls MPI, outside the OpenMP regions.

GPU algorithm (CUDA). The CSR, r, x and the outdegrees stay in GPU memory for the whole run. One iteration launches a contribution kernel, an SpMV kernel and a CUB block reduction for δ and the residual. Only the residual scalar returns to the host. Four SpMV variants were written:

- *thread per row*: one thread loops over one vertex's in-edges;
- *warp per row*: 32 threads split the in-edges of a vertex and combine them with a warp shuffle reduction;
- *degree buckets*: vertices are sorted once into three buckets by in-degree, then handled by one thread (≤ 16 in-edges), one warp (≤ 1024) or one 256-thread block (> 1024);
- `cusparseSpMV`, used only as a reference for what a vendor library achieves. It is not part of our method, in line with the rule against relying on numerical libraries; no BLAS or LAPACK routine is used anywhere.

The single-GPU program stores offsets as 32-bit integers (required by the cuSPARSE reference), so it cannot load graphs with more than 2^{31} edges. The **MPI+CUDA** program uses one rank per GPU, the same edge partition and ghost plan as the CPU code, and 64-bit offsets, so it runs com-friendster (3.6G edges) on one or more GPUs. Its iteration is: contribution and pack kernels on the GPU; copy the packed send buffer to the host; `MPI_Neighbor_alltoallv` and `MPI_Allreduce` on the host; copy the received ghosts to the GPU; SpMV kernel; residual `MPI_Allreduce`.

3 Data sets

Real graphs (SNAP). *web-Google*: a web graph released by Google; small, used for validation and for the data findings. *soc-LiveJournal1*: the friendship graph of an online community, directed;

Table 3. Graphs after preprocessing (duplicate edges and self-loops removed). Density is M/N^2, the fraction of nonzero entries of A.

graph	N	M	M/N	density	dangling	max in-deg	max out-deg
web-Google	875,713	5,105,039	5.8	6.7×10^{-6}	15.6%	6,326	456
soc-LiveJournal1	4,847,571	68,475,391	14.1	2.9×10^{-6}	11.4%	13,905	20,292
com-friendster	65,608,366	3,612,134,270	55.1	8.4×10^{-7}	0%	5,214	5,214
R-MAT `rmat_p`k	131,072 k	$\approx 15.6\,N$	14.8–15.8	10^{-4}–1.7×10^{-7}	41–59%		

518,382 self-loops were removed. *com-friendster*: the friendship graph of an online gaming network. It is undirected, so each undirected edge is stored in both directions, every vertex has equal in- and out-degree and there are no dangling vertices. Preprocessing (`tools/preprocess.cpp`) skips comment lines, renumbers vertex identifiers to $0..N-1$, removes self-loops and duplicate edges, and writes the binary in-edge CSR (64-bit offsets, 32-bit indices) with the outdegree array.

Synthetic graphs (R-MAT). `tools/rmat_gen.cpp` implements the R-MAT generator with the Graph500 parameters $(a, b, c, d) = (0.57, 0.19, 0.19, 0.05)$. For each edge it descends $\lceil \log_2 N \rceil$ levels of the adjacency matrix and at each level picks one of the four quadrants with those probabilities. Edges whose source or target is $\geq N$ are redrawn. Vertex labels are then randomly permuted, so that high-degree vertices are not clustered at small identifiers (which would make a contiguous partition easier than on real data). Self-loops and duplicates are removed. The edges are generated in parallel with one `mt19937_64` generator per OpenMP thread, seeded from the graph seed and the thread number, so a graph is reproducible from (seed, thread count). All graphs were generated with 32 threads. Two families were generated:

- `rmat_p`k for weak scaling: $N = 131{,}072\,k$ vertices and 16 edges per vertex before deduplication, for $k \in \{1, 2, 4, 8, 16, 36, 72, 144, 288, 432, 576, 720\}$, i.e. about 2 million edges per core when run on k cores (seed k);
- `rmat_n20_ef`f for the serial size study: fixed $N = 2^{20}$ and edge factor $f \in \{4, 8, 16, 32, 64\}$.

Sparsity. All graphs are extremely sparse: fewer than 7 nonzeros per million entries of A, and the density falls as the graph grows, because the average degree is bounded while N^2 grows. Sparsity has three consequences used throughout the report:

1. the work per iteration is $\Theta(N + M)$, not $\Theta(N^2)$, and is dominated by irregular reads of x;
2. the degree distributions are heavy-tailed (max in-degree is thousands of times the mean), so equal vertex counts do not give equal work. A vertex partition of soc-LiveJournal1 over 288 ranks has an edge imbalance of 8.62, while the edge partition reaches 1.05 (Table 8);
3. there is no locality to exploit after the random relabelling. The in-edges of a block come from all over the graph, so almost every pair of ranks shares edges and the ghost set of a rank grows with the total N, not only with the size of its own block. This is what limits scaling in Section 7.

4 Experimental setup

Hardware and software. All runs used the NUS Vanda cluster. A CPU node (`cn`) has two Intel Xeon Platinum 8452Y processors (2×36 cores, 2 NUMA domains, 2×67.5 MB L3 cache) and 503 GB of memory. A GPU node has the same CPUs and two NVIDIA A40 GPUs (48 GB, 696 GB/s peak memory bandwidth, 6 MB L2 cache). Software: GCC 13.1 (`-O3 -march=x86-64-v3`), OpenMPI 4.1.8, CUDA 12.4 (`nvcc -O3 -arch=sm_86`). The nodes' RDMA-capable network adapters are Intel `irdma` devices. Section 7.4 shows that this OpenMPI build cannot use them for MPI, so inter-node messages go over TCP. Jobs were submitted without naming a queue (Vanda routes them), always requesting whole 72-core nodes. Threads were pinned with `OMP_PROC_BIND=close`, `OMP_PLACES=cores` and ranks with `-map-by ppr:`k`:node:PE=`t `-bind-to core`.

Measurement. Performance runs execute a fixed number of iterations with tolerance 0: 50 iterations for soc-LiveJournal1 and R-MAT, 10 for com-friendster (one com-friendster iteration takes 25 s serially). Every configuration is run as 1 warm-up and 5 timed repetitions (3 for com-friendster) inside one `mpirun`, after a barrier. The reported time is the median over repetitions of $(\max_{\text{ranks}}$ wall time$)$/iterations. Graph loading, partitioning, ghost-plan construction and host-to-GPU copies are excluded from the iteration time and reported separately. Speedups and efficiencies always compare runs of the same graph and iteration count. Every number, table and figure in this report is generated from the raw CSV files by `tools/make_report.py`.

Table 4. Validation against the serial C++ program. L_1 and L_∞ are the norms of the difference of the two PageRank vectors. web-Google: run to convergence (residual $< 10^{-10}$, 118 iterations for every implementation). soc-LiveJournal1: exactly 50 iterations. MPI+OpenMP on web-Google: 4 ranks × 4 threads.

graph	implementation	L_1	L_∞	same top-100 order
web-Google	MPI+OpenMP, vertex, allgather	3.2e-15	4.8e-18	yes
web-Google	MPI+OpenMP, vertex, ghost	3.2e-15	4.8e-18	yes
web-Google	MPI+OpenMP, edge, allgather	3.3e-15	5.0e-18	yes
web-Google	MPI+OpenMP, edge, ghost	3.3e-15	5.0e-18	yes
web-Google	CUDA, thread per row	3.0e-15	4.4e-18	yes
web-Google	CUDA, warp per row	3.4e-15	8.9e-18	yes
web-Google	CUDA, degree buckets	3.4e-15	9.1e-18	yes
web-Google	CUDA, cuSPARSE	3.4e-15	9.0e-18	yes
web-Google	MPI+CUDA, 1 GPU	3.5e-15	9.1e-18	yes
web-Google	MPI+CUDA, 2 GPUs	3.6e-15	9.2e-18	yes
soc-LiveJournal1	pure MPI, 144 ranks, 2 nodes	1.6e-14	4.4e-18	yes
soc-LiveJournal1	pure MPI, 288 ranks, 4 nodes	1.6e-14	4.4e-18	yes
soc-LiveJournal1	pure MPI, 432 ranks, 6 nodes	1.6e-14	4.4e-18	yes

5 Validation

Correctness was checked at three levels.

1. **Against an independent implementation.** `tools/smoke.py` compares the serial program and the MPI program (4 ranks, all partition and exchange combinations) with NetworkX `pagerank` on hand-made graphs that contain the corner cases: a cycle, dangling vertices, isolated vertices, a graph with no edges (all mass dangling), and partitions with empty blocks. All 52 comparisons pass with $L_1 < 10^{-10}$ and $|\sum_v r_v - 1| < 10^{-12}$. On web-Google the converged serial vector agrees with NetworkX to $L_1 = 1.3 \times 10^{-10}$, with the same top-100 order.
2. **Every parallel variant against the serial program** on web-Google run to convergence (Table 4). All four MPI+OpenMP variants, all four CUDA kernels and MPI+CUDA on one and two GPUs need the same 118 iterations and differ from the serial vector by $L_1 \approx 3 \times 10^{-15}$, i.e. by floating-point rounding only (the summation order differs). The top-100 vertices are identical and in the same order, and the rank vector sums to 1 within 10^{-11}.
3. **At scale.** PageRank vectors of soc-LiveJournal1 after exactly 50 iterations, computed with up to 432 pure-MPI ranks on several nodes, were written to disk and compared with the serial vector (`tools/compare_vec.py`). This checks the ghost plan under the communication patterns used in the scaling runs: the difference is at most $L_1 = 1.6e-14$.

Every timed run additionally reports the final residual and $\sum_v r_v$. Rows with a wrong sum or a different residual would have been visible in the CSV files; none occurred.

> **Key result.** Every MPI+OpenMP, CUDA and MPI+CUDA variant reproduces the serial vector up to floating-point rounding ($L_1 \approx 3 \times 10^{-15}$ on web-Google), and the multi-node runs stay within $L_1 = 1.6e-14$ on up to 432 ranks.

6 Serial runtime and problem size

To vary both dimensions of the sparse matrix, we vary the number of edges M at a fixed number of vertices N (edge factor 4 to 64), and grow N and M together over three orders of magnitude (R-MAT `rmat_p`k, 1.9M to 1.5G edges). The real graphs are also included (Figure 1, Table 5).

The run time grows linearly with M, as the $\Theta(N + M)$ analysis predicts: all points lie close to the slope-1 line in Figure 1a, from 4.5 ms to 9.0 s. The constant, however, depends on where x fits in the memory hierarchy:

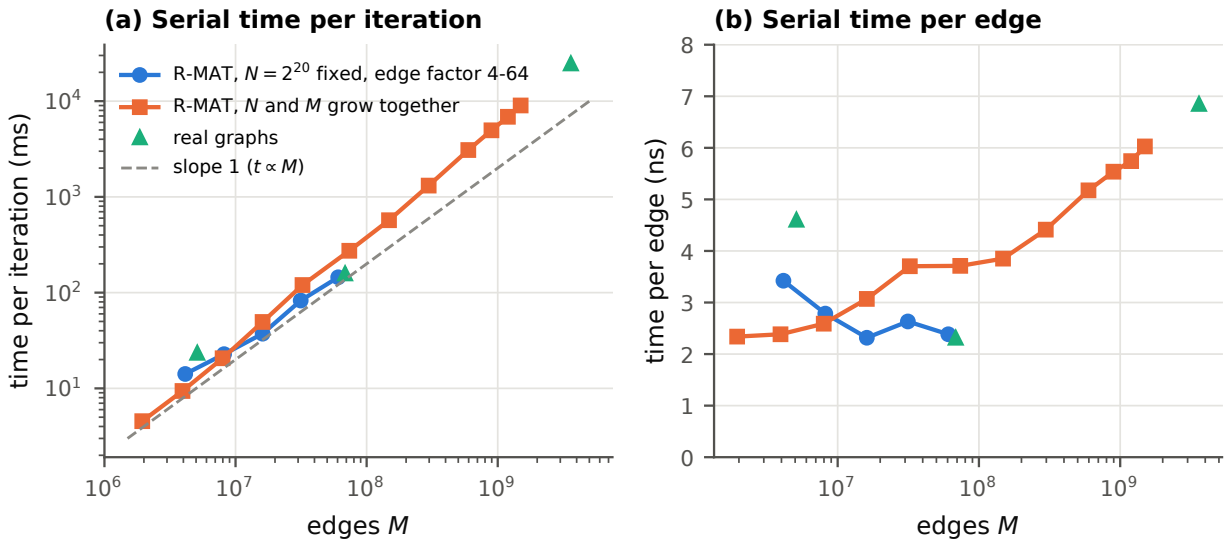


Figure 1. Serial program, one core. (a) Time per iteration against the number of edges for three families. (b) The same time divided by M.

Table 5. Serial time per iteration (median of 3 repetitions of 10 iterations).

family	graph	N (M)	M (M)	M/N	ms / iteration	ns / edge
fixed N	`rmat_n20_ef4`	1.05	4.1	3.9	14.1	3.42
fixed N	`rmat_n20_ef8`	1.05	8.2	7.8	22.8	2.79
fixed N	`rmat_n20_ef16`	1.05	16.1	15.3	37.3	2.32
fixed N	`rmat_n20_ef32`	1.05	31.4	29.9	82.7	2.63
fixed N	`rmat_n20_ef64`	1.05	60.7	57.9	145	2.38
N, M grow	`rmat_p1`	0.13	1.9	14.8	4.54	2.34
N, M grow	`rmat_p2`	0.26	3.9	15.0	9.39	2.38
N, M grow	`rmat_p4`	0.52	8.0	15.2	20.6	2.59
N, M grow	`rmat_p8`	1.05	16.1	15.3	49.4	3.07
N, M grow	`rmat_p16`	2.10	32.4	15.5	120	3.70
N, M grow	`rmat_p36`	4.72	73.8	15.6	274	3.71
N, M grow	`rmat_p72`	9.44	148.1	15.7	571	3.85
N, M grow	`rmat_p144`	18.87	297.3	15.8	1312	4.41
N, M grow	`rmat_p288`	37.75	596.4	15.8	3087	5.18
N, M grow	`rmat_p432`	56.62	895.5	15.8	4958	5.54
N, M grow	`rmat_p576`	75.50	1195.6	15.8	6866	5.74
N, M grow	`rmat_p720`	94.37	1495.6	15.8	9014	6.03
real	`web-Google`	0.88	5.1	5.8	23.5	4.61
real	`soc-LiveJournal1`	4.85	68.5	14.1	159	2.32
real	`com-friendster`	65.61	3612.1	55.1	24748	6.85

- At fixed $N = 2^{20}$ (an 8 MB x vector that fits in the L3 cache), the time per edge *falls* from 3.42 to 2.38 ns as the edge factor grows from 4 to 64. The per-vertex part of the work (the $44N$ bytes of the bandwidth model) is spread over more edges.
- When N and M grow together, the time per edge *rises* from 2.34 to 6.03 ns. Up to about $N = 10^6$ the x vector fits in cache. Beyond $N \approx 1.7 \times 10^7$ ($8N > 135$ MB, the total L3) almost every x_{src} read goes to DRAM.
- The real graphs follow the same rule. soc-LiveJournal1 ($x = 39$ MB, fits in L3) runs at 2.3 ns per edge, com-friendster ($x = 525$ MB) at 6.9 ns per edge, i.e. 24.7 s per iteration.

The serial model is therefore $T \approx M \cdot t_e(N) + c\,N$, with t_e between 2.3 and 7 ns depending on whether $8N$ bytes fit in the cache. This also explains the cache-related effects seen in the parallel

runs: splitting a graph over more ranks shrinks the part of x each rank touches.

> **Key result.** Serial time grows linearly with the number of edges; the number of vertices sets the constant, through whether the $8N$-byte vector x fits in the 135 MB L3 cache (2.3 ns per edge) or not (up to 7 ns).

7 MPI scaling

7.1 Strong scaling

Figures 2–3 and Table 6 show strong scaling from one core to 10 nodes for both large real graphs. Within one NUMA domain (up to 36 cores) the hybrid code is close to ideal. com-friendster reaches $S_{36} = 35.1$ ($E = 0.97$); the slight superlinearity at 2–16 cores is a cache effect (the threads together hold more of x in their caches). soc-LiveJournal1 reaches $S_{36} = 27.3$. Beyond one node the two graphs behave differently:

- **com-friendster** keeps scaling: 421 ms on one node, 278 ms on 2 nodes, 193 ms on 4 nodes and 159 ms on 432 cores, a speedup of 159 over one core and of 2.6× over one node. Efficiency falls from 0.84 (72 cores) to 0.46 (288) and 0.22 (720), where the time stops improving (162 ms).
- **soc-LiveJournal1** is too small to profit from more than one socket. One iteration takes only 6.10 ms on 36 cores, which is less than one exchange of its ghost values between nodes. The time stays between 6 and 9 ms from 36 to 720 cores. The best value, 5.83 ms on 720 cores, is only slightly better than one socket.

Pure MPI is slower than hybrid at every core count above one, and it collapses on several nodes. On soc-LiveJournal1 it reaches 9.84 ms at 144 ranks but 34.22 ms at 288, and 0.3–0.45 s per iteration at 432–720 ranks: 54× slower than hybrid at 432 cores and slower than one core. On com-friendster pure MPI is fastest at 144 ranks (745 ms) and then slows down again, reaching 2215 ms at 720 ranks, 14× the hybrid time. The reasons are analysed next.

7.2 Weak scaling

For weak scaling the graph grows with the core count (Figure 4, Table 7), from 1.9 million edges on one core to 1.5 billion edges on 720 cores. The hybrid time per iteration grows from 2.76 ms to 73.8 ms, a weak efficiency of 0.04. Three effects cause the growth, and they can be separated:

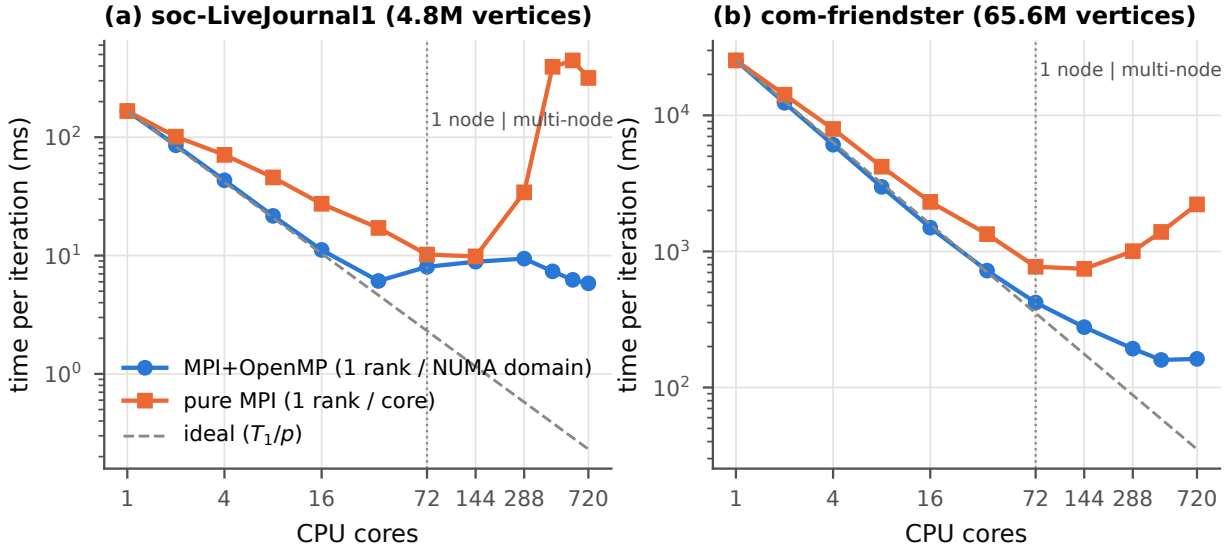


Figure 2. Strong scaling, 1 to 720 cores (10 nodes). Time per iteration of the hybrid layout (one rank per NUMA domain, 36 threads per rank) and of pure MPI (one rank per core) on (a) soc-LiveJournal1 and (b) com-friendster. The dotted line marks one full node.

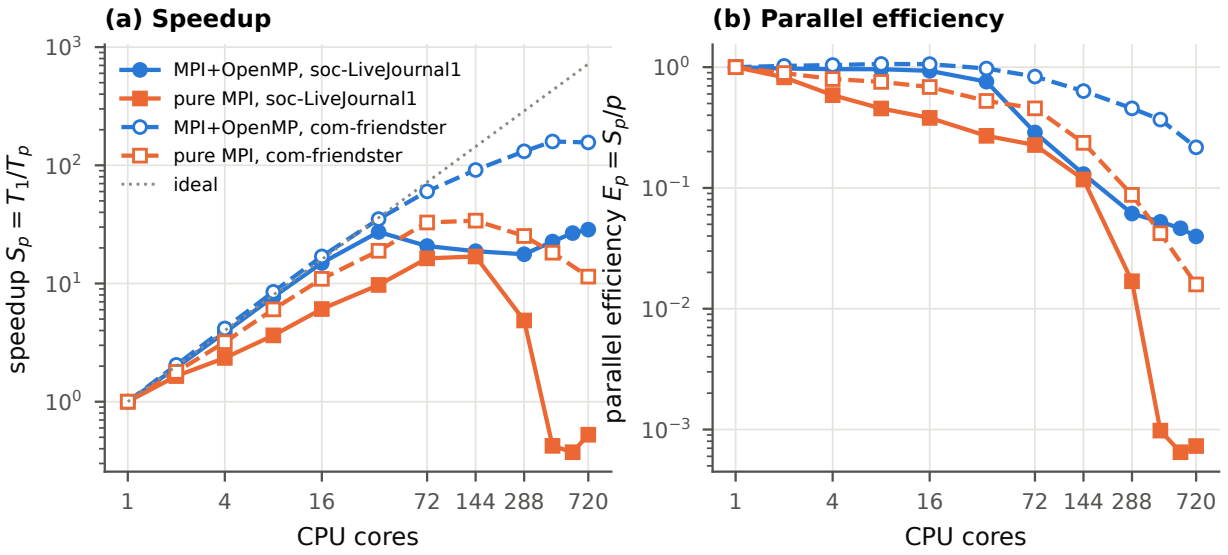


Figure 3. Strong-scaling speedup and parallel efficiency, relative to the same program on one core. Filled markers: soc-LiveJournal1, open markers: com-friendster.

Table 6. Strong scaling: time per iteration T_p (ms), speedup S_p and efficiency E_p. "–": configuration did not complete (Section 7.4).

	soc-LiveJournal1						com-friendster					
	MPI+OpenMP			pure MPI			MPI+OpenMP			pure MPI		
cores	T_p	S_p	E_p	T_p	S_p	E_p	T_p	S_p	E_p	T_p	S_p	E_p
1	166	1	1.00	166	1	1.00	25335	1	1.00	25335	1	1.00
2	85.5	1.9	0.97	101	1.6	0.82	12370	2	1.02	14181	1.8	0.89
4	43.2	3.9	0.96	71	2.3	0.59	6084	4.2	1.04	7940	3.2	0.80
8	21.6	7.7	0.96	45.7	3.6	0.45	2980	8.5	1.06	4194	6	0.76
16	11.2	15	0.93	27.4	6.1	0.38	1495	17	1.06	2313	11	0.68
36	6.1	27	0.76	17.1	9.7	0.27	722	35	0.97	1340	19	0.53
72	8.03	21	0.29	10.2	16	0.23	421	60	0.84	772	33	0.46
144	8.88	19	0.13	9.84	17	0.12	278	91	0.63	745	34	0.24
288	9.42	18	0.061	34.2	4.9	0.017	193	131	0.46	1004	25	0.088
432	7.35	23	0.052	394	0.42	0.00098	159	159	0.37	1391	18	0.042
576	6.24	27	0.046	446	0.37	0.00065	–	–	–	–	–	–
720	5.83	29	0.04	317	0.53	0.00073	162	156	0.22	2215	11	0.016

1. **Memory bandwidth within a node** (1–72 cores, efficiency down to 0.21). One core draws 18 GB/s from memory but 36 cores together only 316 GB/s (STREAM triad, Figure 7): the bandwidth per core drops by half when the socket is full. A bandwidth-bound kernel cannot weak-scale inside a node.
2. **Cache capacity.** The per-core share of edges is constant, but a rank reads x entries from the whole graph. The x vector of `rmat_p36` (38 MB) still fits in L3; that of `rmat_p144` (151 MB) does not. This is the same effect that raises the serial time per edge (Section 6).
3. **Communication across nodes** (72–720 cores). After the random relabelling, each rank's ghost set grows with the total N, so the bytes and the number of partners per rank grow even though the edges per rank stay fixed. From 72 to 720 cores the communication time of the hybrid runs grows from 4.8 to 61 ms per iteration while the computation time stays at 8–16 ms.

Pure MPI stays within a factor of two of the hybrid code up to 144 ranks (38.6 ms). It then jumps to 754 ms at 288 ranks, the same breakdown as in strong scaling.

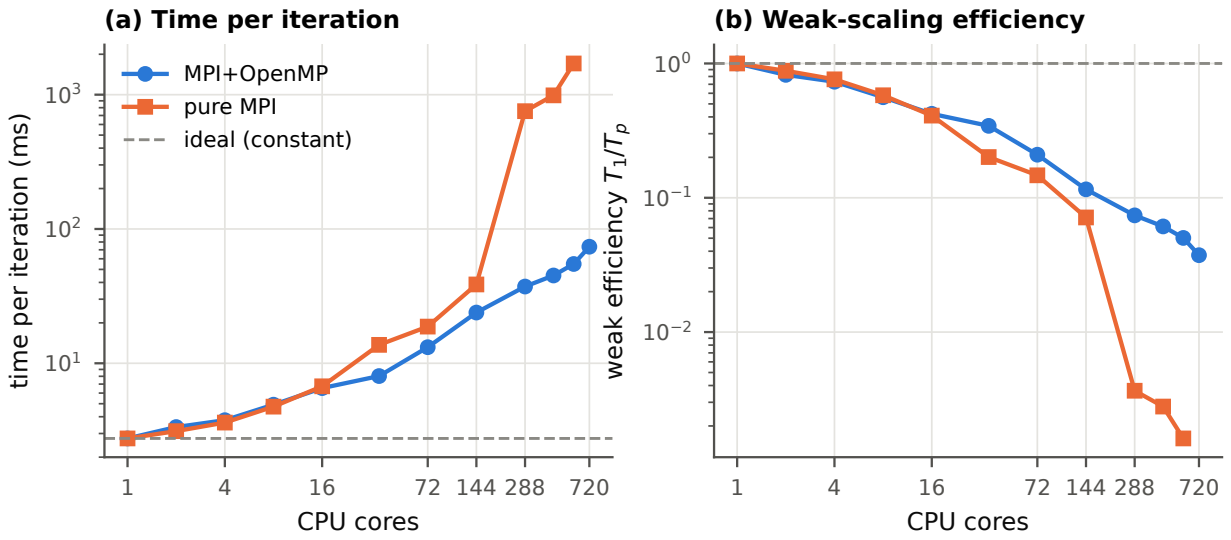


Figure 4. Weak scaling on R-MAT graphs with ≈2M edges per core (`rmat_p`k on k cores; 1.9M edges on 1 core, 1.5G edges on 720 cores). Ideal weak scaling is a constant time.

Table 7. Weak scaling on `rmat_p`k: time per iteration (ms) and weak efficiency T_1/T_p.

			MPI+OpenMP		pure MPI	
cores	N (M)	M (M)	T_p	T_1/T_p	T_p	T_1/T_p
1	0.13	2	2.76	1.00	2.76	1.00
2	0.26	4	3.35	0.82	3.12	0.88
4	0.52	8	3.77	0.73	3.62	0.76
8	1.05	16	4.92	0.56	4.76	0.58
16	2.10	32	6.53	0.42	6.74	0.41
36	4.72	74	8.03	0.34	13.7	0.20
72	9.44	148	13.2	0.21	18.8	0.15
144	18.87	297	23.9	0.12	38.6	0.071
288	37.75	596	37.3	0.074	754	0.0037
432	56.62	895	45.1	0.061	988	0.0028
576	75.50	1196	54.9	0.05	1707	0.0016
720	94.37	1496	73.8	0.037	–	–

7.3 Impact of communication

Communication is the part of the problem that limits scaling. The three MPI operations per iteration have very different costs:

- the two `MPI_Allreduce` calls move one double each. They cost only latency, and they act as global synchronisation points;
- the **ghost exchange** moves all the data. Its volume (Figure 5a) is set by the number of ghost vertices. The hybrid layout sends fewer bytes than pure MPI at the same core count: 2–10× fewer for soc-LiveJournal1 and 6–26× fewer for com-friendster. With 36 threads per rank, the 36 cores share one copy of each ghost value, whereas 36 separate ranks would each receive their own copy. Allgather would be far worse: on 288 ranks it moves 11 GB per iteration for soc-LiveJournal1, against 291 MB with the ghost exchange (Table 8).

The share of time spent in MPI calls (Figure 5b) grows with the number of ranks and exceeds one half for every multi-node configuration of soc-LiveJournal1. The `-split` runs (Figure 6) separate waiting from moving data. For com-friendster on 72 cores, an iteration spends 371.33 ms computing, 19.96 ms waiting at barriers and 49.75 ms moving data. On 720 cores computation has dropped to 39.11 ms, roughly in proportion to the cores, but the data exchange has *grown* to 104.81 ms.

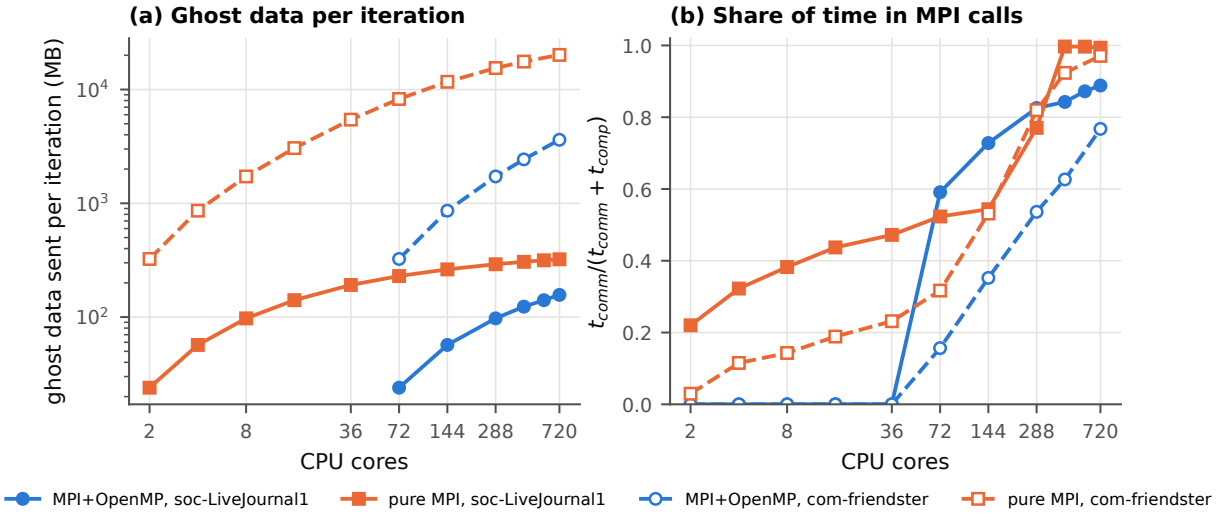


Figure 5. (a) Total ghost data sent per iteration, summed over ranks (MB). (b) Share of the time spent in MPI calls, $t_{comm}/(t_{comm} + t_{comp})$, from the strong-scaling runs.

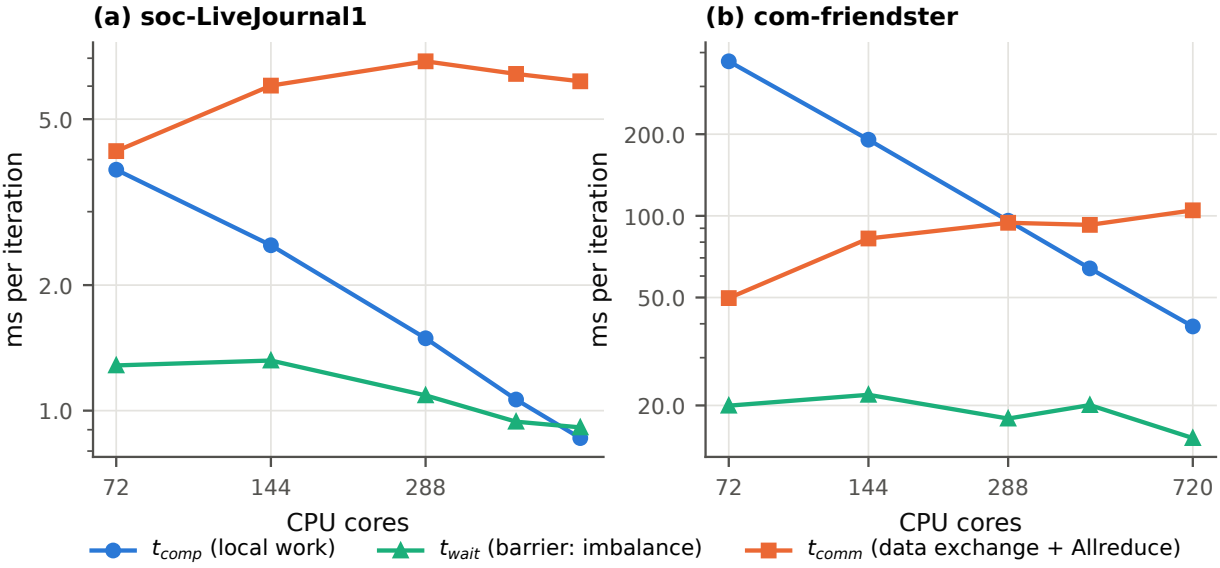


Figure 6. Diagnostic `-split` runs of the hybrid code: computation, barrier wait (load imbalance and synchronisation) and pure communication per iteration, from 1 to 10 nodes, for (a) soc-LiveJournal1 and (b) com-friendster.

The crossover is at about 288 cores, which matches where the strong-scaling efficiency drops below 0.5. The barrier wait stays small (15–22 ms) because the edge partition balances the work: load imbalance is not the limit, communication is.

Even within one node, going from one rank (36 cores) to two ranks (72 cores) makes soc-LiveJournal1 slower (6.10 → 8.03 ms). The two ranks exchange their ghost values through shared memory, and this costs 4.19 ms per iteration, more than the 3.78 ms of computation it saves. On Vanda the single-copy (CMA) mechanism of the shared-memory transport is not permitted, so every message is copied twice through a shared buffer.

7.4 Why pure MPI breaks down: TCP between nodes

The node's RDMA network adapters are Intel `irdma` devices (iWARP/RoCE). The OpenMPI 4.1.8 module on Vanda is built without UCX, and its `openib` BTL is disabled for these devices by default. We forced it on (`-mca btl self,vader,openib -mca btl_openib_allow_ib 1`, experiment E9). Every such run failed at start-up on 2–10 nodes with "No OpenFabrics connection schemes reported

that they were able to be used" followed by "unable to reach each other". Inter-node MPI messages on Vanda therefore go through the kernel TCP stack (`btl tcp`), with its per-message latency and per-connection socket buffers.

With TCP, the cost of the ghost exchange depends strongly on the number of messages, not only on the bytes. In a random-like graph almost every rank has ghost vertices on almost every other rank, so the exchange consists of about $P(P-1)$ messages per iteration: 380 for the 20 ranks of hybrid on 10 nodes, but about 83,000 for 288 pure-MPI ranks and 518,000 for 720. This is what we see:

- at 288 ranks on 4 nodes, the allgather (a ring/recursive-doubling collective with $O(P)$ messages per rank) beats the ghost exchange on com-friendster (682 vs 1045 ms with the edge partition) despite sending 10× more bytes (Table 8);
- pure MPI on soc-LiveJournal1 jumps from 10 ms (144 ranks) to 34 ms (288) and 0.3–0.45 s (432–720): almost all of it is t_{comm} (about 20 s of the 20 s total at 432 ranks);
- with 576 and 720 ranks, every rank opens a TCP connection to almost every rank on another node, about $72 \times 504 \approx 36{,}000$ connections per node at 576 ranks. A run that started less than 60 s after the previous large run failed with "`bind on local address ... Address already in use`": the sockets of the previous run were still in the TCP `TIME_WAIT` state, so the node had no free local port. The same configurations succeeded when started a few minutes later. The failed runs were repeated in a separate job that waits for these sockets to drain before every large run; any that never completed are shown as "–".

The hybrid layout avoids the problem because it uses 36× fewer ranks. It sends fewer bytes, and above all it sends about 36^2 times fewer messages and opens 1,300× fewer TCP connections. On this cluster the MPI+OpenMP design is therefore not just faster but the only one that scales beyond a few nodes. An MPI library with RDMA support (e.g. an HPC-X/UCX build) would lower the per-message cost, but it would not change the P^2 growth of the message count.

Key result. The hybrid code scales com-friendster to a speedup of 159 on 432 cores, while soc-LiveJournal1 is too small to gain beyond one socket. Weak efficiency at 720 cores is 0.04. In both cases the ghost exchange is what grows, and over TCP its cost follows the number of messages ($\approx P^2$), so pure MPI collapses above 144 ranks.

8 MPI+X

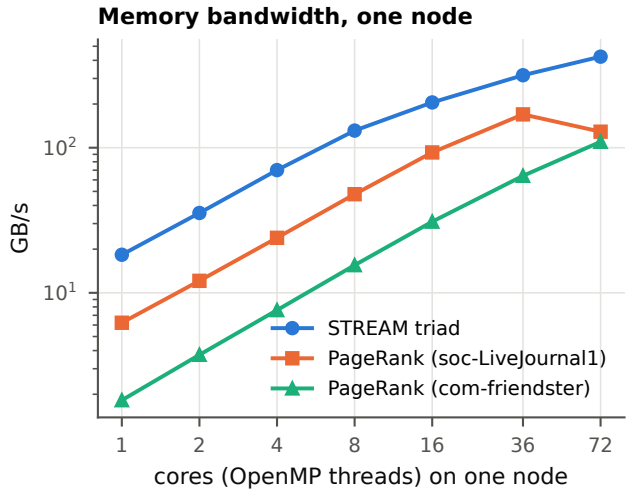


Figure 7. Memory bandwidth on one node: STREAM triad and the effective bandwidth of the PageRank iteration (hybrid code) for 1 to 72 cores.

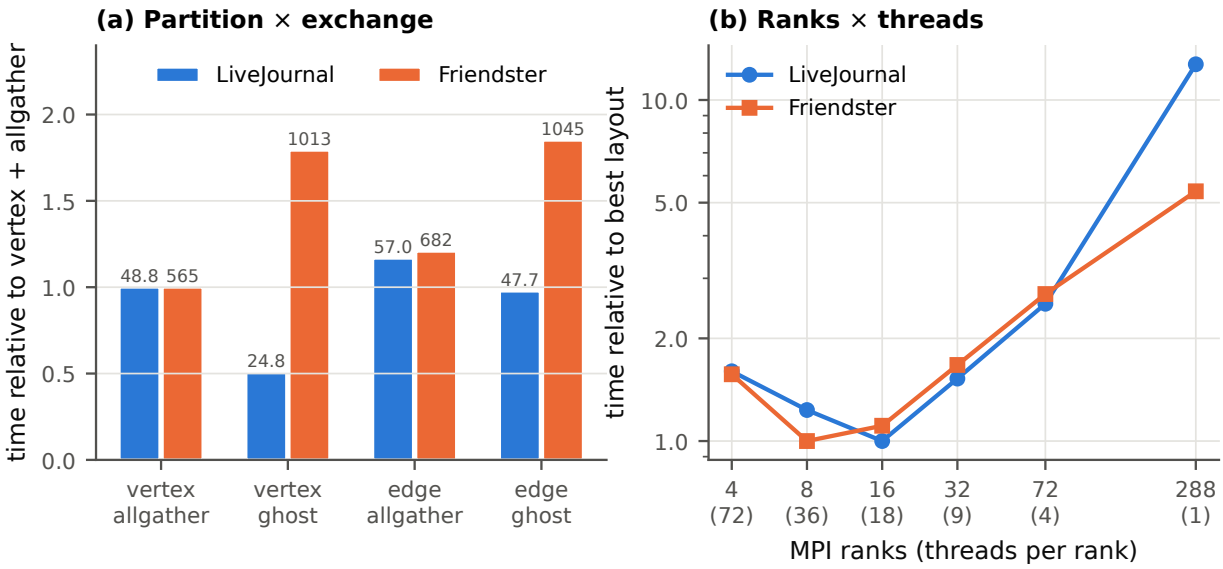


Figure 8. (a) The four partition and exchange combinations on 288 pure-MPI ranks (4 nodes), relative to the vertex partition with allgather; the bar labels give ms per iteration. (b) 288 cores split into different numbers of ranks and threads.

Table 8. Partition and exchange methods on 288 ranks (4 nodes, pure MPI). Times in ms per iteration; the bytes are the total sent per iteration over all ranks.

graph	partition	exchange	T	t_{comp}	t_{comm}	imbalance	MB / iteration
`soc-LiveJournal1`	vertex	allgather	48.8	5.5	48.4	8.62	11130
`soc-LiveJournal1`	vertex	ghost	24.8	4.4	24.6	8.62	256
`soc-LiveJournal1`	edge	allgather	57.0	2.4	56.7	1.05	11130
`soc-LiveJournal1`	edge	ghost	47.7	5.3	47.5	1.05	291
`com-friendster`	vertex	allgather	564.7	234.1	546.7	1.79	150637
`com-friendster`	vertex	ghost	1012.5	262.2	995.2	1.79	15045
`com-friendster`	edge	allgather	682.2	145.0	595.6	1.00	150637
`com-friendster`	edge	ghost	1045.3	194.3	944.1	1.00	15450

8.1 X = OpenMP: threads, ranks and design choices

OpenMP within a node. OpenMP threads parallelise the two vertex loops; the SpMV loop uses a dynamic schedule with chunks of 512 rows, so that threads that hit long rows do not hold up the others. Figure 7 compares the effective bandwidth of the iteration with STREAM. For soc-LiveJournal1 the iteration reaches 170 GB/s on one socket, more than half of STREAM (316 GB/s): x fits in the L3 cache, so most of the irregular reads hit cache. com-friendster reaches 110 GB/s on the full node, a quarter of STREAM (424 GB/s), because each random x read moves a full cache line from DRAM. Both curves bend where STREAM bends: the thread level is limited by memory bandwidth, which is why the hybrid code puts exactly one rank on each socket's memory.

Ranks × threads. On 288 cores (Figure 8b) the best split for both graphs is 8 to 16 ranks with 36 or 18 threads, i.e. one or two ranks per NUMA domain. Fewer ranks (4×72 threads) put threads on the remote socket's memory. More ranks increase the communication, up to 13× the best time for soc-LiveJournal1 with 288 single-thread ranks.

Partition and exchange. Table 8 compares the four combinations on 288 ranks. The edge partition fixes the load imbalance (8.62 → 1.05 on soc-LiveJournal1, 1.79 → 1.00 on com-friendster) and reduces the com-friendster computation time from 262.16 to 194.26 ms. The ghost exchange cuts the bytes by 10–40×. As discussed in Section 7.4, with pure MPI over TCP the message count matters more than the bytes, so on com-friendster the allgather is faster at this rank count. With the hybrid layout the rank count stays small and the ghost exchange is the better choice. All scaling

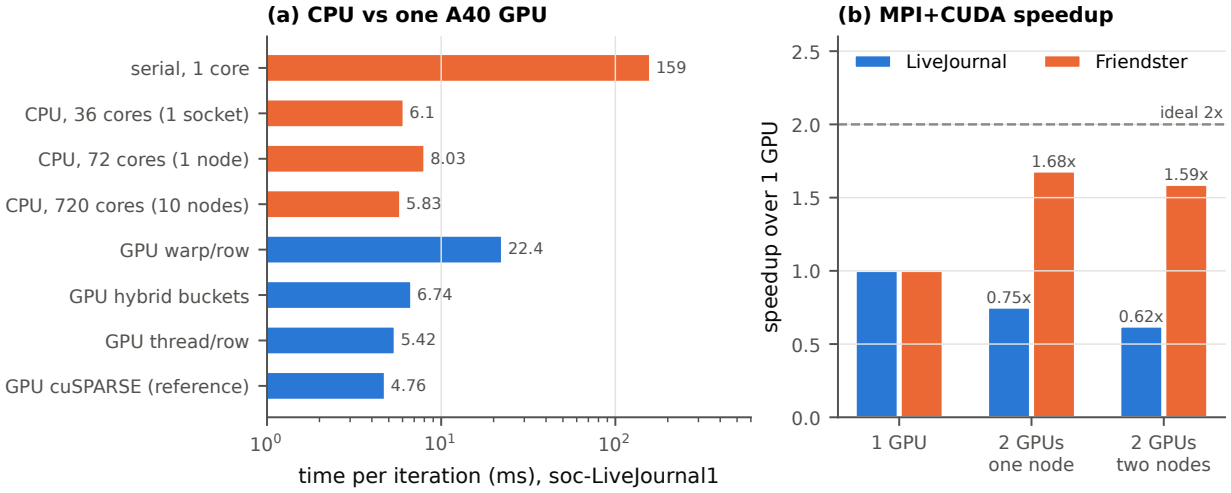


Figure 9. (a) soc-LiveJournal1 time per iteration on CPU cores and on one A40 GPU (four SpMV kernels). (b) MPI+CUDA speedup with two GPUs on one node and on two nodes.

Table 9. Single-GPU kernels on soc-LiveJournal1 (one A40, median of 5×50 iterations): time per iteration, effective bandwidth B and B as a fraction of the 696 GB/s peak.

SpMV kernel	ms / iteration	B (GB/s)	of peak
one thread per row	5.42	187	27%
one warp per row	22.40	45	7%
degree buckets (thread / warp / block)	6.74	151	22%
cuSPARSE SpMV (reference only)	4.76	213	31%

runs therefore use the edge partition with the ghost exchange.

8.2 X = CUDA: one GPU

On one A40 (Table 9, Figure 9a), the simple thread-per-row kernel is the fastest of our kernels (5.42 ms). The warp-per-row kernel is 4× slower (22.40 ms) because the median row has only a handful of in-edges, so most of the 32 lanes idle. The degree-bucket kernel (6.74 ms) helps the few very long rows but pays for a second level of indirection through the bucket lists. cuSPARSE, used only as a reference, reaches 4.76 ms, 12% faster than our best kernel.

Compared with the CPU, one A40 runs soc-LiveJournal1 24× faster than one core but only about as fast as one 36-core socket (5.42 vs 6.10 ms), and 1.5× faster than a full node (8.03 ms). For com-friendster (MPI+CUDA, one GPU) one iteration takes 584 ms, which is *slower* than one CPU node (421 ms). The GPU is *not* limited by its arithmetic. We measured its memory system with `tools/gpu_bw.cu`: a device-to-device copy reaches 565 GB/s and a STREAM triad 542 GB/s, but a random gather $y_i = x_{\mathrm{idx}_i}$, the access pattern of the SpMV, reaches only 113 GB/s. The GPU moves memory in 32-byte sectors and its 6 MB L2 cannot hold the 39 MB x vector, so each 8-byte random read costs a full sector from DRAM. The CPU, in contrast, keeps the whole x of soc-LiveJournal1 in its 135 MB of L3. Our best kernel reaches 187 GB/s of effective bandwidth (27% of the peak). This is above the random-gather probe, presumably because the in-edges of a vertex are stored sorted, so neighbouring reads sometimes share a sector. This is why the GPU gains only a small factor over a CPU socket: both are bound by irregular memory access, and here the GPU's bandwidth advantage over the CPU's cache is small.

8.3 MPI+CUDA: multiple GPUs

The MPI+CUDA program was run with two GPUs on one node and with one GPU on each of two nodes (Figure 9b). On com-friendster, two GPUs give a speedup of 1.68 on one node (584 → 348 ms) and 1.59 across two nodes (368 ms). The computation halves exactly (584 → 297 ms per GPU), and the rest is the ghost exchange through the host: 48 ms of MPI on one node, 68 ms over

TCP between nodes, plus 13 ms of host–device copies. On soc-LiveJournal1 one GPU needs only 4.74 ms per iteration, less than the exchange itself, so two GPUs are *slower* (6.32 and 7.63 ms). This is the same lesson as on the CPU: distributing pays off only when the local work per iteration exceeds the cost of one exchange. Vanda's GPU queue allows at most two GPUs per job, so larger GPU counts could not be tested.

> **Key result.** OpenMP inside a socket is limited by memory bandwidth, so one rank per NUMA domain is the best layout. One A40 GPU matches one 36-core socket on soc-LiveJournal1: its random-gather bandwidth (113 GB/s), not its arithmetic, sets the speed. Two GPUs help only when the local work exceeds one exchange (com-friendster: 1.68×).

9 Discussion

Conclusions about the data. On web-Google the PageRank is highly concentrated: the top 0.1% of pages hold 11.4% of the total rank, the top 1% 27.1% and the top 10% 56.8%. PageRank is strongly but not perfectly correlated with in-degree (Spearman $\rho = 0.85$), and 88 of the 100 highest-ranked pages are also among the 100 pages with most in-links. The others rank high because they are linked from important pages, not from many pages. For example, the page ranked 7th has only the 23rd largest in-degree. The 161,168 pages without in-links all share the minimum score $(1-d)/N + d\delta/N = 2.46 \times 10^{-7}$, about a fifth of the uniform value $1/N$. The iteration converges geometrically: the residual falls by a factor of about 0.85 per iteration, as the damping factor bounds it, and reaches 10^{-10} after 118 iterations.

For the computation, the properties that matter are those of Section 3. The data sets differ mainly in whether the x vector fits in the CPU cache (web-Google and soc-LiveJournal1: yes; com-friendster and the large R-MAT graphs: no), and in the ratio of local work to communication. com-friendster has 55 edges per vertex: on 144 cores it processes 34 edges per received ghost value, against 10 for soc-LiveJournal1, which is why it keeps scaling to 432 cores while soc-LiveJournal1 stops at one node. The heavy-tailed degree distribution makes the edge partition necessary: with a vertex partition, the slowest rank of soc-LiveJournal1 on 288 ranks has 8.6× the average work.

How the problem was parallelised.

- *Between nodes and sockets (MPI):* a 1D row partition by edges, so every rank has the same SpMV work, and a precomputed ghost exchange, so every rank receives only the x values it needs. The partition and the communication pattern are computed once and reused in every iteration.
- *Within a socket (OpenMP):* the vertex loops are split over threads that share the rank's memory, so the 36 cores of a socket share one copy of the ghost values and need no messages among themselves.
- *On the GPU (CUDA):* one thread per row for the SpMV, and the vectors stay on the GPU. Several kernels were compared, and the measured memory probes explain their speed.

Which parts need the most communication. The ghost exchange of x in every iteration needs by far the most communication: all the data volume, and $O(P^2)$ messages in a random-like graph. The two all-reduces carry only one number each, but they synchronise all ranks twice per iteration, which is where the barrier wait in Figure 6 is paid. Loading and partitioning need no communication beyond a broadcast of the bounds, because every rank reads its own slice of the file. In our measurements the ghost exchange takes more time than the computation for every multi-node configuration of soc-LiveJournal1, and for com-friendster from about 288 cores.

Limitations. (i) Inter-node MPI ran over TCP; an RDMA-capable MPI would shift all multi-node curves down, and should help pure MPI most. (ii) We did not reorder the vertices (e.g. by graph partitioning or locality-aware relabelling). Such reordering would reduce the number of ghost vertices and is the most promising way to improve scaling further. (iii) GPU runs are limited to two GPUs per job on Vanda. (iv) Pure MPI at 576 and 720 ranks needs tens of thousands of TCP

connections per node and is close to the port limit of a node; configurations that did not complete are reported as missing, not estimated.

Declaration of AI use

We used an AI assistant (Claude, by Anthropic) during this project for code review and debugging of the job scripts and timing code, for writing the experiment scripts (`jobs/final`), the bandwidth probes and the analysis and plotting scripts, and for drafting and editing parts of this report. All experiments were designed, run and checked by us on Vanda. Every number in the report comes from our own measurement files, and we reviewed all code and text.

Source code and reproduction

The repository contains the source (`src/`), the preprocessing and generator tools (`tools/`), the PBS job scripts used for every result (`jobs/final/`), the raw result files (`results_final/`) and the script that builds all figures and tables (`tools/make_report.py`). `README.md` describes how to build, download the data, submit the jobs and regenerate this report.

References

[1] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank citation ranking: bringing order to the web. Stanford InfoLab technical report, 1999.

[2] J. Leskovec, A. Krevl. SNAP Datasets: Stanford large network dataset collection. http://snap.stanford.edu/data, 2014.

[3] D. Chakrabarti, Y. Zhan, C. Faloutsos. R-MAT: a recursive model for graph mining. SIAM International Conference on Data Mining, 2004.

[4] Graph500 benchmark specification, https://graph500.org.

[5] J. D. McCalpin. Memory bandwidth and machine balance in current high performance computers. IEEE TCCA Newsletter, 1995.